



Universidad Autónoma Metropolitana
Unidad Cuajimalpa

Protocolo de Proyecto Terminal I

**Plataforma distribuida de comunicación y alertas comunitarias
para entornos universitarios (Noti Uam)**

Alumno: Aaron Rodrigo Ramos Reyes
Carrera: Sistemas y Tecnologías de Información
Trimestre: 26-P
Asesor: Dr. Guillermo Monroy Rodriguez
Universidad: UAM Cuajimalpa

Ciudad de México, México
2026

Plataforma distribuida de comunicación y alertas comunitarias para entornos universitarios (Noti Uam)

Aaron Rodrigo Ramos Reyes
Universidad Autónoma Metropolitana, Unidad Cuajimalpa
Ciudad de México, México
aaron.r.ramos@cuam.uam.mx

RESUMEN

La comunicación dentro de las universidades suele estar fragmentada entre múltiples plataformas externas (Foross, redes sociales, correo institucional), lo que provoca baja visibilidad de eventos académicos, desinformación y dificultad para descubrir canales de interés. Este proyecto terminal presenta **Noti Uam**, una plataforma distribuida de comunicación y alertas comunitarias orientada a entornos universitarios. El sistema se basa en una arquitectura distribuida *publish-subscribe* (pub-sub) que permite la creación de canales temáticos, suscripción abierta y notificaciones en tiempo real. Se propone implementar el backend con Spring Boot y Apache Kafka, utilizar PostgreSQL y Redis para persistencia y caché, y desarrollar una aplicación móvil multiplataforma con Flutter. El objetivo general es diseñar y construir una solución escalable, usable y accesible que mejore la difusión de información académica, cultural y comunitaria dentro de la UAM Cuajimalpa. El presente documento detalla el planteamiento del problema, los fundamentos teóricos, el estado del arte, los objetivos, la hipótesis, la metodología incremental y el cronograma de trabajo para los tres trimestres de Proyecto Terminal.

CCS CONCEPTS

• **Human-centered computing** → **Collaborative and social computing systems and tools**; • **Security and privacy** → *Management and access control*.

KEYWORDS

notificaciones académicas, sistemas distribuidos, publish-subscribe, Apache Kafka, Flutter, usabilidad

1. INTRODUCCIÓN

1.1. Contexto de la comunicación en entornos universitarios

La comunicación efectiva dentro de las instituciones de educación superior constituye un pilar fundamental para el desarrollo de actividades académicas, culturales y sociales. Diariamente, estudiantes, profesores y personal administrativo generan y consumen una gran cantidad de información: avisos de clase, cambios de horario, fechas de examen, convocatorias a eventos, formación de grupos de estudio, difusión de talleres, entre muchos otros mensajes. Sin embargo, la forma en que esta información se distribuye actualmente en la mayoría de las universidades —incluyendo la Universidad Autónoma Metropolitana (UAM)— dista de ser óptima.

Diversos estudios han documentado que los estudiantes recurren de manera masiva a plataformas externas como *WhatsApp*, *Facebook*, *Instagram*, *Telegram* o el correo electrónico institucional para

comunicarse y mantenerse informados [1, 7]. Estas herramientas, si bien son de fácil acceso y uso cotidiano, no fueron diseñadas para gestionar la comunicación estructurada y escalable de una comunidad universitaria. Como señala Gutiérrez (2017), el uso de aplicaciones de mensajería instantánea en el ámbito académico se ha dado de manera espontánea y no regulada, lo que genera tanto oportunidades como problemas de fragmentación y desinformación.

1.2. Problemática identificada

La dependencia de plataformas no especializadas produce múltiples efectos negativos:

- **Fragmentación de la información:** Los anuncios importantes suelen dispersarse en decenas de grupos de WhatsApp, páginas de Facebook o cadenas de correo. Un estudiante que no pertenece a un grupo específico puede perderse avisos relevantes sobre su curso o su carrera.
- **Dificultad para descubrir contenido:** En plataformas como WhatsApp, los grupos son cerrados y requieren una invitación. Esto impide que los alumnos conozcan la existencia de comunidades de estudio, eventos culturales o asesorías abiertas [1].
- **Riesgo de desinformación:** Al no existir mecanismos de verificación o moderación, cualquier usuario —incluso ajeno a la institución— puede publicar contenido falso o irrelevante. Esto afecta la credibilidad de la comunicación institucional.
- **Baja efectividad del correo institucional:** Aunque formal y moderado, el correo electrónico suele ser revisado con poca frecuencia por los estudiantes, y los sistemas de notificaciones asociados son lentos o poco visibles.

Bucheli (2020) encontró, en un estudio realizado en la Universidad Autónoma del Estado de Hidalgo, que más del 80 % de los estudiantes utilizan WhatsApp para comunicarse con sus compañeros y profesores, pero al mismo tiempo reportan sentir “sobrecarga de información” y “estrés por la mezcla de temas personales y académicos”. Esta situación se replica en muchas universidades públicas mexicanas, incluida la UAM Cuajimalpa.

1.3. Insuficiencia de las soluciones existentes

Algunas universidades han implementado sistemas propios de gestión de aprendizaje (como Moodle, Blackboard, Ubicua o plataformas personalizadas). Estos sistemas, si bien son útiles para entregar tareas y consultar calificaciones, no están diseñados para la difusión ágil de notificaciones en tiempo real ni para la creación de canales comunitarios abiertos. Por otra parte, aplicaciones como *Telegram* ofrecen canales públicos y notificaciones, pero su adopción no es homogénea en toda la comunidad universitaria y siguen siendo herramientas externas sin moderación o barreras de entrada

adecuadas implicando una vulnerabilidad a la desinformación u agentes externos.

En el ámbito de los sistemas distribuidos y las notificaciones en tiempo real, existen arquitecturas probadas como el modelo *publish-subscribe* (pub-sub) que permite desacoplar a los productores de información de los consumidores, garantizando escalabilidad y entrega eficiente de eventos [12, 13]. Sin embargo, muy pocas plataformas universitarias han adoptado este tipo de arquitecturas para la comunicación cotidiana.

1.4. Propuesta de solución: Noti Uam

Ante este panorama, se propone el desarrollo de **Noti Uam**, una plataforma distribuida de comunicación y alertas comunitarias específicamente orientada a entornos universitarios. La plataforma se basa en una arquitectura *publish-subscribe* (pub-sub) que permite:

- La creación de **canales temáticos** (por asignatura, por actividad deportiva, por área de interés) visibles para toda la comunidad.
- La **suscripción voluntaria y abierta** a dichos canales, sin necesidad de invitaciones previas.
- El envío de **notificaciones en tiempo real** a los dispositivos móviles de los usuarios suscritos.
- Un sistema de autenticación seguro mediante JWT, que garantice que solo miembros de la comunidad universitaria puedan publicar (evitando spam o desinformación externa).

La plataforma será desarrollada como una **aplicación móvil multiplataforma** usando *Flutter*, dado que los dispositivos móviles son el principal medio de acceso a internet entre los estudiantes. El *backend* se implementará con *Spring Boot* y *Apache Kafka* para la gestión de eventos, mientras que el almacenamiento de datos se realizará con *PostgreSQL* y *Redis* para optimizar el rendimiento. Todo el sistema se contenerizará con *Docker* para facilitar su despliegue y escalabilidad.

1.5. Contribución del trabajo

La principal contribución de este proyecto es proporcionar una solución tecnológica adaptada a las necesidades específicas de comunicación de la comunidad universitaria de la UAM Cuajimalpa, que:

- Reduce la fragmentación informativa al centralizar los avisos en una sola aplicación.
- Favorece el descubrimiento abierto de actividades académicas, recreativas y culturales.
- Mejora la eficiencia de la comunicación institucional al ofrecer un canal directo y visible para los estudiantes.
- Aumenta la seguridad y confiabilidad de la información al restringir la publicación a usuarios autenticados.
- Ofrece notificaciones inmediatas basadas en una arquitectura distribuida robusta (pub-sub).
- Incorpora principios de Interacción Humano-Computadora (IHC) para garantizar usabilidad y accesibilidad.

Además, desde el punto de vista académico, el proyecto permite aplicar y demostrar competencias en sistemas distribuidos, bases de datos, desarrollo móvil y metodologías de diseño centrado en el usuario.

1.6. Estructura del protocolo

El presente protocolo se organiza de la siguiente manera: después de esta introducción, el **Marco Teórico** (sección 2) expone los conceptos fundamentales de sistemas distribuidos, el modelo pub-sub, las tecnologías seleccionadas y los principios de IHC. El **Estado del Arte** (sección 3) revisa soluciones similares existentes y las compara. La sección 4 detalla los **objetivos** general y específicos. La sección 5 plantea la **hipótesis** de trabajo. La **Metodología** (sección 6) describe el enfoque incremental y las fases de desarrollo. Finalmente, la sección 7 presenta el **cronograma** tentativo y la sección 8 lista las referencias bibliográficas.

2. MARCO TEÓRICO

El presente marco teórico expone los conceptos fundamentales necesarios para comprender y justificar las decisiones tecnológicas y de diseño de la plataforma **Noti Uam**. Se abordan, en primer lugar, los principios de los sistemas distribuidos y, en particular, el modelo de arquitectura *publish-subscribe* (pub-sub). Posteriormente, se describen las tecnologías específicas seleccionadas para el desarrollo del *backend*, la base de datos, la capa de mensajería y la aplicación móvil. Finalmente, se introducen los principios de Interacción Humano-Computadora (IHC) y los mecanismos de autenticación y seguridad.

2.1. Sistemas distribuidos: principios y paradigmas

Un **sistema distribuido** es aquel en el que componentes de software y hardware ubicados en equipos de red conectados se comunican y coordinan mediante el paso de mensajes, con el objetivo de compartir recursos y ofrecer servicios de manera unificada a los usuarios [4]. Tanenbaum y Van Steen (2023) definen un sistema distribuido como “una colección de computadoras independientes que aparece ante sus usuarios como un solo sistema coherente”.

Las propiedades deseables de los sistemas distribuidos incluyen:

- **Transparencia:** ocultar al usuario y al programador la complejidad de la distribución (ubicación, concurrencia, fallos, etc.).
- **Escalabilidad:** capacidad de crecer en número de nodos o usuarios sin degradar el rendimiento.
- **Tolerancia a fallos:** continuar operando, incluso si algunos componentes fallan.
- **Desacoplamiento en tiempo y espacio:** los productores y consumidores de información no necesitan estar activos simultáneamente ni conocerse entre sí.

La última propiedad es particularmente relevante para sistemas de notificaciones como *Noti Uam*, donde los emisores de avisos (estudiantes, profesores) y los receptores (suscriptores) operan de manera asíncrona y pueden estar desconectados en distintos momentos.

2.2. Arquitectura publish-subscribe (pub-sub)

El modelo **publish-subscribe** es un estilo arquitectónico para sistemas de comunicación basada en eventos [13]. En este modelo, los productores (*publishers*) generan eventos o mensajes y los publican en un *broker* o intermediario sin conocer a los suscriptores. Los consumidores (*subscribers*) expresan su interés en ciertos tipos

de eventos (normalmente mediante filtros o canales) y el *broker* se encarga de entregar los mensajes a todos los suscriptores relevantes.

Las ventajas principales del modelo pub-sub son:

- **Desacoplamiento espacial:** publicadores y suscriptores no necesitan saber la dirección o identidad de los demás.
- **Desacoplamiento temporal:** no es necesario que ambas partes estén activas al mismo tiempo; el *broker* almacena y reenvía los mensajes cuando el suscriptor esté disponible.
- **Escalabilidad:** se pueden añadir nuevos nodos publicadores o suscriptores sin modificar la lógica central.

Estas características hacen de pub-sub una arquitectura ideal para sistemas de alertas y notificaciones en tiempo real, como el propuesto en este proyecto. Según Tarkoma (2012), las implementaciones modernas de pub-sub pueden manejar millones de eventos por segundo con latencias del orden de milisegundos.

2.3. Componentes tecnológicos del backend

2.3.1. Apache Kafka. **Apache Kafka** es una plataforma distribuida de *streaming* de eventos diseñada originalmente por LinkedIn. Se ha convertido en un estándar de facto para construir tuberías de datos en tiempo real y aplicaciones reactivas [9]. Kafka implementa un modelo de *log distribuido* que combina las ventajas de pub-sub con almacenamiento duradero y alta capacidad de *throughput*.

En *Noti Uam*, Kafka actuará como el *broker* central de eventos. Los productores (API REST del backend) publicarán mensajes en *topics* (equivalentes a canales temáticos) y los consumidores (servicio de notificaciones push) se suscribirán a esos *topics* para enviar alertas a los dispositivos móviles. La naturaleza distribuida de Kafka permite escalar horizontalmente para atender a toda la comunidad universitaria.

2.3.2. Spring Boot. **Spring Boot** es un marco de trabajo de Java que simplifica la creación de aplicaciones basadas en el ecosistema Spring, proporcionando configuración automática y servidores embebidos (como Tomcat) [11]. Se utiliza ampliamente en la industria para desarrollar microservicios y *backends* RESTful.

En la plataforma propuesta, Spring Boot será el encargado de:

- Exponer los endpoints de la API REST para gestionar usuarios, canales y publicaciones.
- Integrarse con Kafka mediante la plantilla *KafkaTemplate* para publicar eventos.
- Implementar la lógica de autenticación y autorización mediante JWT.
- Servir de puente entre la aplicación móvil y los servicios internos.

2.3.3. PostgreSQL y Redis. El almacenamiento de datos persistentes se realizará con **PostgreSQL**, un sistema de gestión de bases de datos relacional y de código abierto, conocido por su fiabilidad, concurrencia y cumplimiento de ACID [6]. Se utilizará para almacenar usuarios, canales, suscripciones y publicaciones históricas.

Por otro lado, **Redis** es un almacén de estructura de datos en memoria que actúa como base de datos, caché y *message broker* ligero. En *Noti Uam*, Redis se empleará para:

- Cachear consultas frecuentes (por ejemplo, los canales más populares) y reducir la carga sobre PostgreSQL.

- Gestionar información temporal, como sesiones activas y tokens de notificaciones push.
- Almacenar contadores de eventos no leídos para cada usuario.

La combinación de PostgreSQL (persistencia) y Redis (rendimiento) es común en sistemas distribuidos con requisitos de baja latencia [12].

2.3.4. Contenerización con Docker. **Docker** permite empaquetar aplicaciones y sus dependencias en contenedores ligeros y portátiles. Cada servicio (Spring Boot, Kafka, PostgreSQL, Redis) se ejecutará en su propio contenedor, definiendo un archivo *docker-compose.yml* que orqueste la red y el orden de inicio. Esto facilita el despliegue tanto en entornos de desarrollo como en producción, garantizando la reproducibilidad del sistema.

2.4. Aplicación móvil multiplataforma: Flutter

Flutter es un kit de desarrollo de interfaz de usuario creado por Google que permite compilar aplicaciones nativas para iOS, Android, Web y escritorio desde una sola base de código en el lenguaje Dart [10]. Su arquitectura basada en *widgets* reactivos y su alto rendimiento (sin puente JavaScript) lo hacen adecuado para aplicaciones que requieren animaciones fluidas y notificaciones en tiempo real.

En *Noti Uam*, Flutter permitirá:

- Desarrollar simultáneamente las versiones para Android e iOS, llegando a la mayoría de los estudiantes.
- Integrar notificaciones push mediante *Firebase Cloud Messaging* (FCM) o *OneSignal*.
- Consumir la API REST del backend mediante el paquete *http dio*.
- Implementar una interfaz moderna, accesible y centrada en la usabilidad, siguiendo los principios de IHC.

2.5. Interacción Humano-Computadora (IHC)

La Interacción Humano-Computadora es una disciplina que estudia el diseño, evaluación e implementación de sistemas interactivos para el uso humano [5]. Uno de sus objetivos centrales es mejorar la **usabilidad**, definida por la norma ISO 9241-11 como “el grado en que un producto puede ser utilizado por usuarios específicos para alcanzar metas específicas con efectividad, eficiencia y satisfacción en un contexto de uso”.

Para *Noti Uam*, los principios de IHC guiarán:

- El diseño de la navegación: permitir a los usuarios descubrir canales fácilmente, suscribirse y publicar avisos en pocos pasos.
- La accesibilidad: cumplir con pautas como contraste suficiente, tamaños de texto ajustables y compatibilidad con lectores de pantalla (accesibilidad móvil).
- La retroalimentación: notificaciones visuales y hápticas confirmatorias de acciones.
- La prevención de errores: validación de formularios, mensajes claros y deshacer acciones.

Se realizarán pruebas de usabilidad con estudiantes reales durante el desarrollo, aplicando métodos como el *System Usability Scale* (SUS) o pruebas de tareas con captura de tiempos.

2.6. Seguridad y autenticación: JWT

La plataforma debe garantizar que solo los miembros autenticados de la comunidad universitaria puedan publicar contenido (aunque cualquier persona pueda leer los canales públicos si así se configura). Para ello, se implementará autenticación basada en tokens JWT (*JSON Web Tokens*) [8].

El flujo típico es:

1. El usuario inicia sesión con credenciales institucionales (correo UAM y contraseña, o integración con SSO de la universidad).
2. El backend valida las credenciales y genera un token JWT firmado que contiene el identificador del usuario y sus roles.
3. El cliente (app móvil) envía el token en el encabezado *Authorization* de cada petición.
4. El backend verifica la firma y los permisos antes de procesar la solicitud.

Los tokens JWT son autónomos (no requieren almacenamiento en servidor), lo que facilita la escalabilidad horizontal. Su tiempo de expiración será corto (por ejemplo, 1 hora) y se complementará con un *refresh token* para mejorar la experiencia de usuario.

2.7. Síntesis del marco teórico aplicado a Noti Uam

La combinación de los conceptos expuestos permite diseñar *Noti Uam* como un sistema distribuido, desacoplado, escalable y seguro, orientado a la alta usabilidad. La siguiente tabla resume la correspondencia entre cada capa tecnológica y los principios teóricos:

Cuadro 1: Correspondencia entre componentes y fundamentos teóricos

Componente	Fundamento teórico
Apache Kafka	Modelo publish-subscribe, desacoplamiento temporal/espacial
Spring Boot	Arquitectura RESTful, contenedor de servicios
PostgreSQL + Redis	Persistencia ACID + caché en memoria
Flutter	Desarrollo multiplataforma, IHC, notificaciones push
JWT	Autenticación stateless, seguridad
Docker	Contenerización, portabilidad, entornos reproducibles

Este marco teórico sustenta las decisiones técnicas que se detallan en la metodología y justifica la viabilidad del proyecto desde una perspectiva académica y profesional.

REFERENCIAS

[1] María Guadalupe Bucheli. 2020. *WhatsApp como herramienta de comunicación en estudiantes universitarios: caso de la Universidad Autónoma del Estado de Hidalgo*. Reporte técnico. Universidad Autónoma del Estado de Hidalgo.

[2] Josiah L. Carlson. 2013. *Redis in Action*. Manning Publications, Shelter Island, NY.

[3] Jhonatan Carrera and Mariana López. 2023. *Implementation of a push notification system for academic events using Flutter and NestJS*. Master's thesis. Universidad Técnica del Norte, Ibarra, Ecuador.

[4] George Coulouris, Jean Dollimore, and Tim Kindberg. 2011. *Distributed Systems: Concepts and Design* (5th ed.). Addison-Wesley, Boston, MA.

[5] AMexCOMP (Asociación Mexicana de Computación). 2018. *Interacción Humano-Computadora y Aplicaciones en México*. Editorial AMexCOMP, México D.F.

[6] PostgreSQL Global Development Group. 2024. PostgreSQL 16 Documentation. <https://www.postgresql.org/docs/16/>. Documentación oficial de PostgreSQL.

[7] Laura Gutiérrez and otros. 2017. Uso de WhatsApp en el ámbito académico universitario: ventajas y desventajas. *Revista de Educación a Distancia (RED)* 17, 54 (2017), 1–23.

[8] M. Jones, J. Bradley, and N. Sakimura. 2015. *JSON Web Token (JWT)*. RFC 7519. Internet Engineering Task Force (IETF).

[9] Jay Kreps, Neha Narkhede, and Jun Rao. 2017. Apache Kafka: A Distributed Streaming Platform. <https://kafka.apache.org/documentation/>. Documentación oficial de Apache Kafka.

[10] Google LLC. 2024. Flutter Documentation. <https://docs.flutter.dev/>. Documentación oficial de Flutter.

[11] Pivotal Software. 2023. Spring Boot Reference Guide. <https://docs.spring.io/spring-boot/docs/current/reference/html/>. Documentación oficial de Spring Boot.

[12] Andrew S. Tanenbaum and Maarten Van Steen. 2023. *Distributed Systems: Principles and Paradigms* (4th ed.). Prentice Hall, Upper Saddle River, NJ.

[13] Sasu Tarkoma. 2012. *Publish/Subscribe Systems: Design and Principles*. John Wiley & Sons, Chichester, UK.